LAB 7
FUZZ TESTING FOR INPUT VALIDATION BUGS

Software Testing Laboratory

Name	Manasi Manoj Jadhav
SRN	PES1UG24CS259
Section	E

1.Objective

This lab aims to find issues with a python application using fuzzing and the Hypothesis property based testing library. The application contains three functions: parse_member_id(), total_fine() and shelf_code(). Issues will be found with input validation and robustness and then fixed using defensive input validation. The tests will then be run again.

2. Tools and Environment

- Programming language: Python 3.14.6
- Testing framework: pytest 9.2.0
- Fuzzing / property-based testing library: Hypothesis 6.170.15
- Editor and terminal: Visual Studio Code with PowerShell
- Application file: processor.py
- Test file: test_processor.py

3. Application Functions

The application processes member IDs, calculates library fines from comma-separated day values, and generates a shelf code from a book title.

- parse_member_id(member_id): separates the member type and numeric identifier.
- total_fine(days_csv): calculates the fine for each value greater than 14 days, at the rate of 2 per excess day.
- shelf_code(title): returns the uppercase initials of the first two words in a title.

4. Procedure and Observations

Step 1 – Run the Original Application

The original processor.py program was executed using representative valid inputs. The member ID STU-1023 was parsed as ('STU', 1023), the day values 10,20,30 produced a total fine of 44, and the title Data Structures produced the shelf code DS.

```
PS C:\Users\hp> cd "C:\Users\hp\Desktop\Library-Fines-Software-Testing-Lab\Lab 7"
PS C:\Users\hp\Desktop\Library-Fines-Software-Testing-Lab\Lab 7> python processor.py
==== Running parse_member_id ====
Enter a member ID (e.g. STU-1023): STU-1023
Member Type and Number: ('STU', 1023)

==== Running total_fine ====
Enter days kept for each book as CSV (e.g. 10,20,30): 10,20,30
Total Fine: 44

==== Running shelf_code ====
Enter a book title with at least two words: Data Structures
Shelf Code: DS
PS C:\Users\hp\Desktop\Library-Fines-Software-Testing-Lab\Lab 7>
```

Figure 1. Execution of the original application with valid sample inputs.

Step 2 – Apply Hypothesis Fuzz Testing

I used hypothesis to test the three application functions with generated text inputs. I tested them using a variety of values including invalid and empty inputs. The first time I ran my tests two passed but one failed. The failure was with the shelf_code function. This resulted in an IndexError being raised as there were no words in the title.

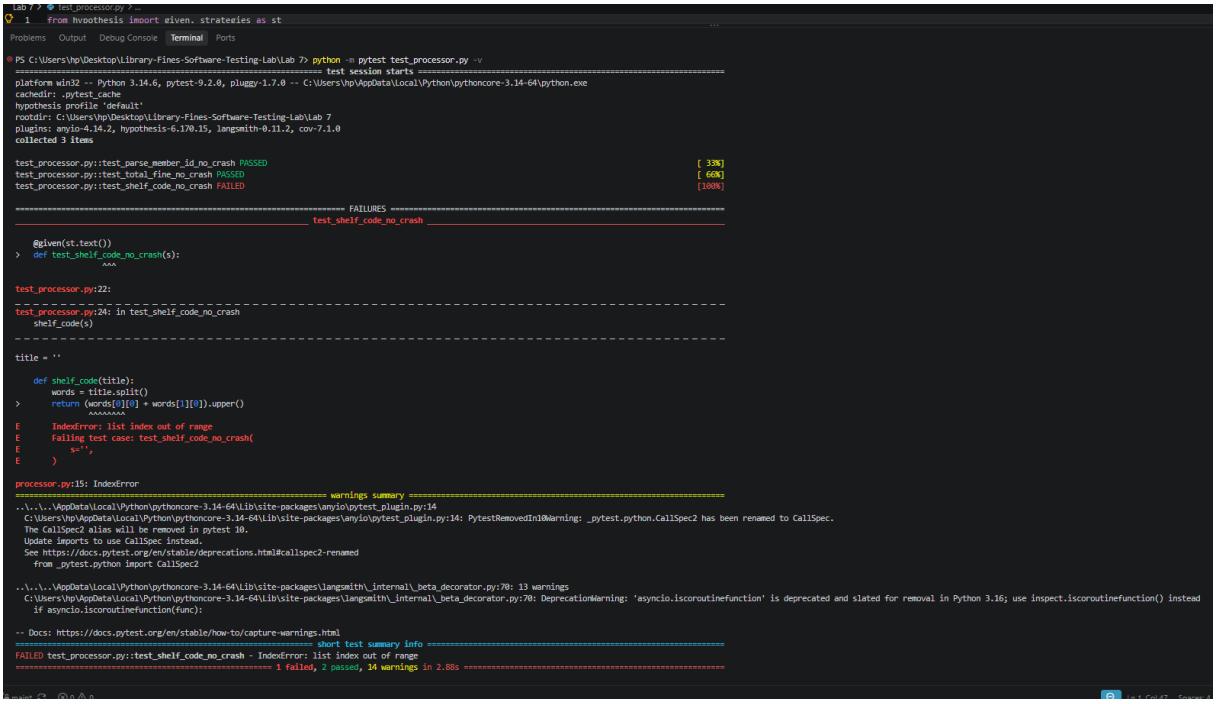


Figure 2. Initial fuzz test run: two tests passed and shelf_code failed for an empty title.

These test cases also produced None values and highlighted errors raised due to the AttributeError as the original functions tried to run .split() on a non string. This error indicated that type and format checking

should be introduced into the functions.

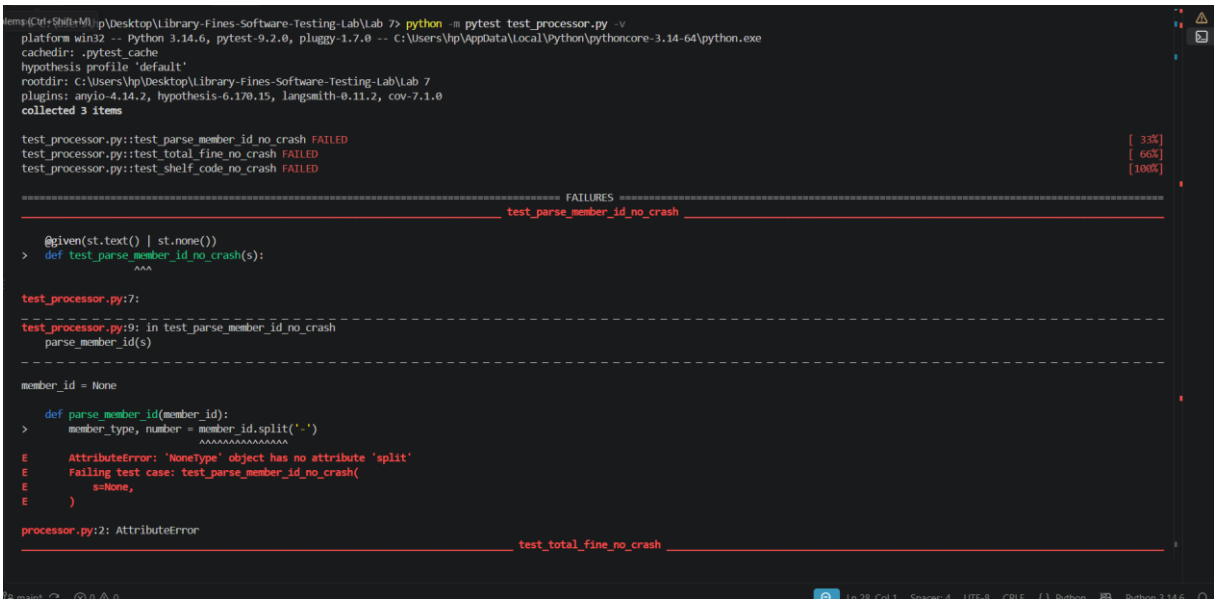


Figure 3. Additional fuzz-generated failure involving None input.

Step 3 – Fix the Application

Improved with defensive validation. The functions check that inputs are strings. Check for correct formatting of member ID. Handles incorrect inputs for integers. Checks that day is not a negative number. Ensures there are at least two words in the book title before retrieving the initials. If incorrect inputs are provided a ValueError will be raised with a message to tell the user why.

The corrected program was then run with valid sample inputs. It successfully parsed STU-1023, calculated a fine of 44 for 10,20,30, and generated DS for Data Structures.

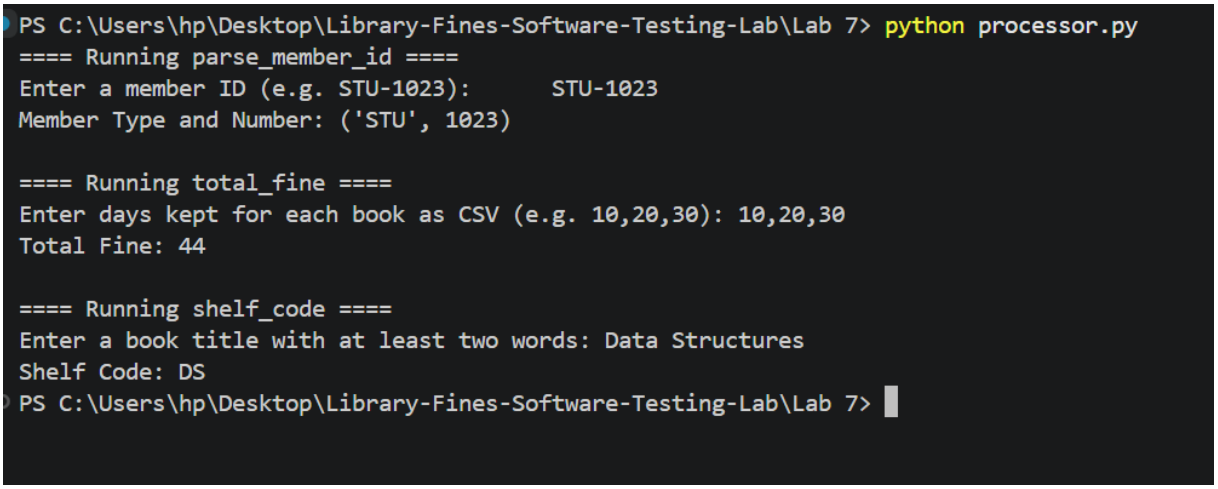


Figure 4. Successful execution of the corrected application.

Step 4 – Re-run the Tests

We ran the tests in Hypothesis again after editing our processor.py file. We ran them using the command python -m pytest test_processor.py -v. All three tests now passed. This meant that our test inputs were not causing any unexpected errors anymore.

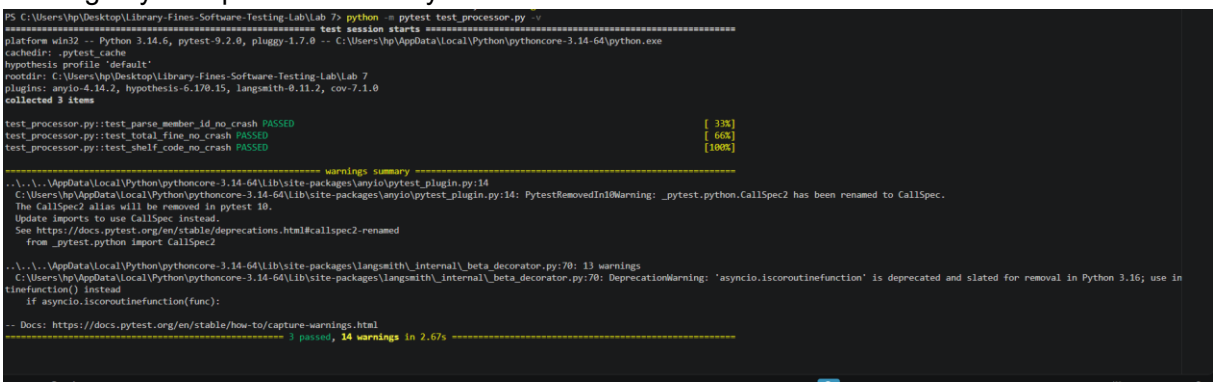


Figure 5. Final pytest result: all 3 tests passed.

5. Result

Fuzzing found several issues with the input handling of the original application. This included IndexError due to empty title and AttributeError due to None inputs. Defensive validation was added to the three functions. The application now works correctly with valid inputs. The Hypothesis/pytest script ran to completion with 3 passing tests.